Proyecto Lights Out

Álgebra Aplicada

Integrantes

Felipe Heredia ci: 5124050-2

Jorge Méndez ci: 5349342-0



24 de octubre de 2026

Resumen

En este trabajo usamos álgebra lineal para resolver el juego Lights Out, en el que hay que apagar todas las luces de un tablero. Primero analizamos qué pasa al presionar una posición y mostramos que alcanza con representar los movimientos mediante ceros y unos. Después planteamos un sistema de ecuaciones módulo 2 y programamos su resolución en Python con eliminación gaussiana. En el tablero de 3×3 de la consigna, el programa encontró las dos pulsaciones esperadas. También probamos tableros de 2×2 y 4×4 y encontramos ejemplos con una sola solución, con varias soluciones y sin solución. Las seis pruebas automatizadas terminaron correctamente. El trabajo muestra cómo pasar de las reglas de un juego a un modelo matemático y comprobar los resultados mediante un programa.

1. Introducción

Lights Out parece un juego sencillo: hay luces encendidas y apagadas y el objetivo es dejarlas todas apagadas. La dificultad está en que, cuando se presiona una luz, también cambian las que están arriba, abajo, a la izquierda y a la derecha, si existen. Por eso, una pulsación puede arreglar una parte del tablero y cambiar otra.

Este tipo de juego puede resolverse con álgebra lineal (Anderson y Feil, 1998). La propuesta de este trabajo es usar álgebra lineal para decidir qué posiciones presionar. Nos interesa entender por qué el juego se puede escribir como un sistema de ecuaciones y comprobar que ese método funciona en Python.

En particular, buscamos justificar cómo se representan las pulsaciones, construir el sistema para el ejemplo de 3×3, resolverlo mediante eliminación gaussiana módulo 2 y probar el programa con tableros que tengan una, varias o ninguna solución.

El informe presenta primero los conceptos matemáticos necesarios, luego explica cómo se construyó el modelo y se implementó el programa, y finalmente muestra los resultados obtenidos, su análisis y las conclusiones del trabajo.

2. Marco teórico

2.1. Cálculos módulo 2

Como cada luz solo puede estar apagada (0) o encendida (1), trabajamos con dos valores. Las sumas se hacen módulo 2:

$$0+0=0, \qquad 0+1=1, \qquad 1+0=1, \qquad 1+1=0.$$

La última igualdad es clave: si una luz cambia dos veces, vuelve al estado inicial. En términos matemáticos, trabajamos en $\mathbb{F}_2$, el cuerpo de dos elementos.

2.2. Vectores y sistemas lineales

Para un tablero de $n \times n$ hay n^2 posiciones. Podemos usar un vector

$$x = (x_1, x_2, \ldots, x_{n^2}),$$

donde $x_i = 1$ significa que presionamos esa posición y $x_i = 0$ que no la presionamos. Así, el problema se expresa como

$$Ax = b \pmod 2.$$

La matriz A indica qué luces cambia cada pulsación, x indica las pulsaciones y b contiene el tablero inicial. Resolver el sistema significa encontrar movimientos que apaguen todas las luces.

2.3. Eliminación gaussiana binaria

La eliminación gaussiana simplifica un sistema mediante operaciones entre filas, como se explica en Lay (2012, sección 1.2). En este caso, usamos sumas módulo 2:

$$F_i \leftarrow F_i + F_j \pmod 2.$$

En Python se pueden realizar con el operador XOR (`^`). Si aparece una fila de ceros con un 1 en el término independiente, el sistema no tiene solución. Si quedan variables libres y el sistema es compatible, puede haber más de una solución.

3. Metodología

3.1. Cómo representamos las pulsaciones

No hace falta presionar una misma posición más de una vez. Si la presionamos dos veces, tanto esa luz como sus vecinas cambian dos veces y vuelven a estar como antes. Si una posición se presiona tres veces, el efecto es el mismo que presionarla una vez.

Tampoco importa el orden de las pulsaciones. Los cambios se suman módulo 2 y esa suma es conmutativa: $a + b = b + a$. Por eso, para describir una solución alcanza con indicar cuáles posiciones se presionan, sin guardar el orden.

Numeramos las posiciones de izquierda a derecha y de arriba hacia abajo. Para asociar la componente x_i con la fila y la columna usamos

$$i - 1 = nq + r, \qquad 0 \leq r < n,$$

de modo que x_i corresponde a la fila $q + 1$ y la columna $r + 1$. Esta numeración permite pasar de una componente del vector a su posición en el tablero.

3.2. Sistema del tablero de 3×3

Trabajamos con el ejemplo dado:

$$B = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad X = \begin{pmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{pmatrix}.$$

Cada ecuación corresponde a una luz. Incluimos las variables de las posiciones que pueden cambiarla: la propia luz y sus vecinas ortogonales. El resultado del lado derecho es 1 si la luz estaba encendida y 0 si estaba apagada.

Por ejemplo, la esquina superior izquierda solo cambia si presionamos a_{11}, a_{12} o a_{21}. Como inicialmente está apagada, su ecuación es $x_1 + x_2 + x_4 = 0$. En cambio, la posición superior central está encendida y recibe cambios de a_{11}, a_{12}, a_{13} y a_{22}, por lo que su ecuación es $x_1 + x_2 + x_3 + x_5 = 1$.

Siguiendo el mismo criterio, las luces de la esquina superior derecha y del centro de la fila izquierda dan $x_2 + x_3 + x_6 = 0$ y $x_1 + x_4 + x_5 + x_7 = 1$, respectivamente. Para la luz central se consideran sus cuatro vecinas y ella misma: $x_2 + x_4 + x_5 + x_6 + x_8 = 1$. La del centro a la derecha depende de x_3, x_5, x_6, x_9; las de la fila inferior dependen, de izquierda a derecha, de (x_4, x_7, x_8), (x_5, x_7, x_8, x_9) y (x_6, x_8, x_9). Como solo la última está encendida,

los términos independientes son 0, 0 y 1.

El sistema completo es:

$$\begin{aligned}
x_1 + x_2 + x_4 &= 0,\\
x_1 + x_2 + x_3 + x_5 &= 1,\\
x_2 + x_3 + x_6 &= 0,\\
x_1 + x_4 + x_5 + x_7 &= 1,\\
x_2 + x_4 + x_5 + x_6 + x_8 &= 1,\\
x_3 + x_5 + x_6 + x_9 &= 0,\\
x_4 + x_7 + x_8 &= 0,\\
x_5 + x_7 + x_8 + x_9 &= 0,\\
x_6 + x_8 + x_9 &= 1.
\end{aligned}$$

Todas las ecuaciones se calculan módulo 2.

3.3. Implementación en Python

El programa separa el trabajo en funciones. Una revisa que el tablero sea cuadrado y contenga solo 0 y 1, y luego construye A y b. Otra aplica la eliminación gaussiana a la matriz aumentada $[A \mid b]$.

Para obtener los pivotes usamos únicamente sumas de filas módulo 2, como pide el trabajo. Si el pivote es 0, buscamos una fila inferior con un 1 y la sumamos a la fila actual. No necesitamos intercambiar filas. Las sumas se hacen con XOR; por ejemplo,

$$(1,0,1,1) + (1,1,0,1) = (0,1,1,0) \quad (\text{mód } 2).$$

Si la reducción llega a una contradicción $0 = 1$, la función devuelve `None`: ese tablero no se puede resolver. Si hay variables libres, las dejamos en 0 para encontrar una solución particular. Esto no asegura la menor cantidad posible de pulsaciones.

La función `resolver_lights_out(tablero)` reúne los pasos anteriores. Para comprobar la respuesta usamos `verificar_solucion(tablero, solucion)`, que revisa $Ax = b$, y `aplicar_pulsaciones(tablero, solucion)`, que simula los movimientos. También usamos `posiciones_pulsadas(solucion,n)` para mostrar las coordenadas de las luces que hay que tocar. Los scripts se entregan por separado, junto con el PDF.

El código del programa y las pruebas automatizadas se entregan junto con este informe. También se pueden consultar en el repositorio:

`https://github.com/FelipeH99/Lightsout`

4. Resultados

4.1. Comprobación del ejemplo de la consigna

La solución del tablero de 3×3 es

$$x = (0, 0, 0, 0, 1, 0, 0, 0, 1).$$

Solo se presionan a_{22} y a_{33}. Al reemplazar $x_5 = x_9 = 1$ y las demás variables por 0 en las nueve ecuaciones, sus lados izquierdos quedan, módulo 2:

$$(0, 1, 0, 1, 1, 0, 0, 0, 1),$$

que coincide con el tablero inicial leído por filas. Por lo tanto, las nueve ecuaciones se cumplen. Al simular las pulsaciones el tablero termina así:

$$B_f = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

La Figura 1 muestra el registro de la ejecución de este ejemplo en Python.

Registro de ejecucion - tablero 3 x 3

```
EJECUCIÓN REAL: TABLERO 3 x 3
Tablero inicial:
  0 1 0
  1 1 0
  0 0 1
Vector solución: [0, 0, 0, 0, 1, 0, 0, 0, 1]
Posiciones a pulsar: [(2, 2), (3, 3)]
Tablero final:
  0 0 0
  0 0 0
  0 0 0
Verificación: True
```

Figura 1: Registro de ejecución del tablero de 3×3: pulsaciones calculadas, tablero final y verificación.

4.2. Pruebas del programa

Probamos los ejemplos iniciales y algunos casos especiales, incluyendo situaciones con soluciones múltiples o incompatibles, como las analizadas en estudios sobre Lights Out (Berman et al., 2021). La Tabla 1 resume los resultados registrados en las ejecuciones.

En el tablero 2×2 apagado no hay que presionar nada. Para el 2×2 completamente encendido, la solución encontrada fue presionar las cuatro posiciones. En el tablero 4×4

Cuadro 1: Resultados de las pruebas realizadas

Caso	Rango de A	Tipo de solución	¿Se verificó?
3×3 de la consigna	9	Única	Sí
2×2 apagado	4	Única	Sí
2×2 encendido	4	Única	Sí
3×3 apagado	9	Única	Sí
4×4 generado	12	Varias	Sí
4×4 incompatible	12	Ninguna	No aplica
4×4 apagado	12	Varias	Sí

generado, el programa también obtuvo una solución válida, aunque no única.

4.3. Un tablero sin solución

Probamos un tablero de 4×4 con una sola luz encendida en la esquina inferior derecha:

$$B = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Su matriz de coeficientes tiene rango 12, pero el sistema es incompatible. Durante la reducción aparece una fila equivalente a

$$(0, 0, \ldots, 0 \mid 1),$$

que exigiría que $0 = 1$. Por eso el programa devolvió `None`. Este ejemplo muestra que no todas las configuraciones iniciales se pueden resolver.

4.4. Un tablero con varias soluciones

También estudiamos el tablero 4×4 completamente apagado. Como A tiene rango 12 y hay 16 incógnitas, quedan cuatro variables libres. Una solución es no presionar nada:

$$x^{(1)} = (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0).$$

Pero también se verificó esta otra:

$$x^{(2)} = (0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0).$$

Las dos dejan el tablero apagado. Como hay cuatro variables libres, este sistema tiene $2^4 = 16$ soluciones posibles.

4.5. Pruebas automatizadas

Las pruebas se ejecutaron con `unittest`. Incluyeron casos básicos, tableros incompatibles, varias soluciones, entradas inválidas y una revisión exhaustiva de los tableros de 1×1 y 2×2. El registro final fue:

```
Ran 6 tests in 0.002s
OK
```

La Figura 2 muestra el registro completo de las pruebas automatizadas.

Registro de pruebas automatizadas

```
test_4x4_generado (test_lights_out.PruebasLightsOut.test_4x4_generado) ... ok
test_casos_basicos (test_lights_out.PruebasLightsOut.test_casos_basicos) ... ok
test_exhaustivo_1x1_2x2 (test_lights_out.PruebasLightsOut.test_exhaustivo_1x1_2x2) ... ok
test_incompatible (test_lights_out.PruebasLightsOut.test_incompatible) ... ok
test_invalidas (test_lights_out.PruebasLightsOut.test_invalidas) ... ok
test_multiples_soluciones (test_lights_out.PruebasLightsOut.test_multiples_soluciones) ... ok

----------------------------------------------------------------------
Ran 6 tests in 0.002s

OK
```

Figura 2: Registro de las seis pruebas automatizadas realizadas con `unittest`.

Las seis pruebas terminaron correctamente. Esto respalda los resultados observados, aunque no reemplaza una demostración formal de que el programa funcione para cualquier tamaño.

5. Discusión

A partir de las pruebas realizadas, vimos que el programa obtiene soluciones que, al aplicarlas al tablero, permiten apagar todas las luces en los casos compatibles que probamos. El ejemplo de 3×3 fue útil para comprobar el resultado, ya que pudimos comparar la solución del programa con los cálculos realizados a mano.

También vimos por qué importa el rango. En los tableros de 2×2 y 3×3, A tiene rango completo, así que cada configuración inicial tiene una única solución. En los de 4×4, el rango es 12 y hay 16 incógnitas: según el estado inicial, el sistema puede no tener solución o tener varias.

La ventaja del programa es que automatiza la construcción de las ecuaciones y su resolución sin usar una biblioteca que resuelva el sistema por nosotros. Su principal límite es

que, si hay varias soluciones, devuelve una de ellas, pero no busca necesariamente la que necesita menos pulsaciones. Esa sería una mejora interesante para continuar el trabajo.

6. Conclusiones

Conclusiones

Para resolver el juego, vimos que se puede representar el tablero con un vector de ceros y unos, donde cada posición indica si una luz está apagada o encendida. No es necesario presionar una misma posición más de una vez, ya que al presionarla dos veces se vuelve al estado inicial. Además, el orden en que se presionan las luces no cambia el resultado. A partir de esto, podemos representar el problema como un sistema de ecuaciones $Ax = b$ en $\mathbb{F}_2$. Para el tablero de 3×3 de la consigna, planteamos las nueve ecuaciones y comprobamos que presionando a_{22} y a_{33} se logra apagar todas las luces.

Al probar el programa, pudimos resolver todos los tableros compatibles que usamos para las pruebas mediante eliminación gaussiana módulo 2. En cambio, para el tablero de 4×4 que tenía encendida solamente la esquina inferior derecha, el programa devolvió `None`, indicando que no tenía solución. Esto se relaciona con el rango de la matriz A: para los tableros de 2×2 y 3×3, la matriz tiene rango completo y la solución es única. En el caso de 4×4, el rango es 12, por lo que puede haber tableros sin solución y otros que tengan varias. Por ejemplo, para el tablero con todas las luces apagadas, existen 16 soluciones.

Una limitación del programa es que, cuando hay varias soluciones posibles, devuelve una de ellas, pero no necesariamente la que requiere menos pulsaciones. También hay que tener en cuenta que las pruebas se realizaron solamente con tableros de hasta 4×4, por lo que no podemos asegurar que el programa funcione correctamente para cualquier tamaño. Como posible mejora, se podría modificar para encontrar la solución que necesite menos pulsaciones y analizar para qué tamaños de tablero la matriz A tiene rango completo.

Referencias

Anderson, M., & Feil, T. (1998). Turning lights out with linear algebra. *Mathematics Magazine, 71*(4), 300–303. https://doi.org/10.1080/0025570X.1998.11996658

Berman, A., Borer, F., & Hungerbühler, N. (2021). Lights out on graphs. *Mathematische Semesterberichte, 68*, 237–255. https://doi.org/10.1007/s00591-021-00297-5

Lay, D. C. (2012). *Álgebra lineal y sus aplicaciones* (4.ª ed.). Pearson Educación.